DESAFIO VERX · DOCUMENTAÇÃO DE ARQUITETURA

Evolução da Arquitetura

Objetivo deste documento

Este documento separa claramente:

- o que a solução já implementa hoje;
- quais gaps estão abertos;
- quais evoluções fazem sentido sem reescrever o raciocínio arquitetural.

Limitações atuais assumidas

As limitações abaixo não anulam a qualidade da solução para o case, mas precisam estar explícitas:

1. a publicação da outbox ainda não tem estratégia robusta de claim/lock para múltiplas instâncias;
2. o consumidor não implementa uma política operacional formal de retry e DLQ;
3. a idempotência concorrente do consumidor ainda segue um modelo simples;
4. os contextos compartilham o mesmo banco físico;
5. a observabilidade é básica;
6. não existe pipeline automatizado de geração dos PDFs a partir dos Markdown.

Gaps assumidos e enquadramento

Gap	Enquadramento
Concorrência do publisher	limitação atual aceitável para o escopo
Retry e DLQ	evolução futura recomendada
Observabilidade madura	evolução futura recomendada
Separação física por contexto	evolução futura recomendada
Testes com infraestrutura real dentro da suíte	melhoria importante, mas não bloqueante para o case
Pipeline de geração de PDFs	melhoria de produtividade e governança documental

Resolução arquitetural prevista para cada gap atual

Esta seção conecta explicitamente os gaps atuais a uma resposta arquitetural futura. O objetivo não é afirmar que esses pontos já estão implementados hoje, e sim mostrar que a arquitetura proposta já tem um caminho claro para absorvê-los sem mudar de direção conceitual.

Gap atual	Risco atual	Solução prevista na evolução	Fase recomendada
Publicação concorrente da outbox	um mesmo evento pode ser publicado mais de uma vez em múltiplas instâncias	introduzir estratégia de claim/lock do evento antes da publicação, ou worker dedicado com coordenação de ownership	Fase 2
Ausência de retry e DLQ	falhas persistentes de processamento ficam sem tratamento operacional robusto	adicionar DLQ, política de retry, visibilidade de falha e trilha de reprocessamento	Fase 2
Idempotência concorrente simples no consumidor	corridas concorrentes ainda dependem de uma abordagem simples	endurecer o controle concorrente com desenho de persistência mais defensivo e tratamento explícito de conflito	Fase 2
Observabilidade básica	operação fica sem métricas, tracing e alarmes arquiteturalmente maduros	adicionar métricas, tracing distribuído, dashboards, alertas e SLOs de processamento	Fase 3
Banco físico compartilhado entre contextos	reduz isolamento operacional e autonomia por domínio	separar fisicamente a persistência do ledger e da projeção de saldo	Fase 2
Testes sem foco maior em infraestrutura real e cenários operacionais	boa cobertura lógica, menor confiança operacional	adicionar validação com fila e banco reais, cenários de falha e baseline de desempenho	Fase 2
Contrato de evento pouco central na narrativa	integração pode parecer implícita em vez de arquitetural	consolidar o evento como contrato versionado e eixo oficial entre contextos	Fase 2
NFRs sem governança operacional madura	risco de ambiguidade entre o que existe e o que é esperado	formalizar SLOs, indicadores operacionais e limites aceitos por contexto	Fase 3
Pipeline manual de geração dos PDFs	risco de divergência entre fonte textual e artefato final	automatizar exportação dos Markdown e dos diagramas para PDF	Fase 3

Como interpretar corretamente esta evolução

Os gaps atuais estão tratados na arquitetura futura de três formas:

1. **endurecimento operacional da mesma abordagem**

Exemplo: outbox, idempotência do consumidor, retry e DLQ.

1. **maior isolamento entre contextos**

Exemplo: separação física de bancos e escala independente.

1. **governança e observabilidade de nível mais alto**

Exemplo: métricas, tracing, SLOs e automação documental.

Em outras palavras, a evolução da arquitetura não muda a linha mestra da solução. Ela torna a mesma direção mais robusta, observável e operável.

Roadmap de evolução sugerido

Fase 1: MVP produtizável

Objetivo:

- manter a simplicidade do case;
- garantir que a execução local e a narrativa arquitetural estejam corretas;
- documentar honestamente trade-offs e limitações.

Prioridades:

- README executivo alinhado aos docs;
- docs em Markdown como fonte de verdade;
- diagramas coerentes com a implementação;
- validação executável do código;
- ajuste de infraestrutura local.

Fase 2: Escala controlada

Objetivo:

- tornar a operação mais robusta sem mudar o desenho conceitual principal.

Prioridades:

- separar banco físico por contexto;
- reforçar o publisher da outbox com claim/lock;
- endurecer a idempotência concorrente do consumer;
- adotar DLQ e políticas de retry;

- criar métricas de backlog e latência de processamento;
- tornar o contrato `entry-created` uma interface de integração ainda mais explícita e governada.

Fase 3: Maturidade operacional

Objetivo:

- levar a arquitetura para um nível mais próximo de ambiente produtivo com governança e observabilidade.

Prioridades:

- tracing distribuído;
- dashboards e alarmes;
- SLOs de processamento assíncrono;
- analytics downstream;
- segurança mais forte e gestão de segredos;
- pipelines de validação e documentação automatizados.

Continuidade da linha arquitetural

O ponto forte desta solução é que a evolução pode acontecer preservando os mesmos princípios centrais:

- `Ledger` continua dono da escrita transacional;
- `Daily Balance` continua dono da leitura agregada;
- o contrato `entry-created` continua sendo o elo principal;
- idempotência continua sendo obrigatória nas bordas;
- outbox continua sendo o mecanismo de confiabilidade entre banco e mensageria.

Ou seja, as evoluções desejadas não exigem jogar fora a base do case. Elas amadurecem a mesma direção arquitetural.

Relação com os diagramas existentes

O diagrama de transição já existente em:

- transition-architecture.drawio

deve ser interpretado como mapa de evolução da solução, e não como fotografia fiel do estado implementado hoje.

Essa distinção precisa permanecer clara em qualquer PDF, README ou material de apresentação.